How did I come up with the code?

Pseudocode:

First do a system call to open the file and read the first 1000 bits and store it into the buffer. After you read the file, initialize a sum that will store the sum and initialize a num that will be added to the sum.

Have to parse through every character in the buffer, if you find a null terminator, finalize the sum. Check to see if in the file is a number and if it is a digit, convert the ascii string to the ascii number.

Once you find the null terminator, that means you are through the whole file and so have to print the sum. Convert the integer sum to a string and print out all the lines to send the message.

The code:

```
section .data
    pathname db "randomInt100.txt", 0
    newline db 10, 0
    sum_msg db "Sum: ", 0

section .bss
    buffer resb 1000   ; Buffer to store file content
    sum resd 1         ; Variable to store the sum
    num resd 1         ; Temporary storage for the current number
    sum_str resb 12    ; String to hold sum (max 10 digits + newline)

section .text
    global _start

_start:
    ; Open the file
    mov eax, 5 ; move 5 into register eax for sys open
    mov ebx, pathname ; move the path into ebx
    mov ecx, 0 ; set ecx to read only
    int 0x80

    mov ebx, eax     ; Store file descriptor

    mov eax, 3 ; move 3 into register eax for sys read
    mov ecx, buffer
```

```
    mov edx, 1000
    int 0x80

    mov edx, eax     ; Store bytes read

    ; Initialize sum and num to 0
    mov dword [sum], 0
    mov dword [num], 0
    mov esi, buffer

parse_loop:
    ; Read character
    mov al, [esi]
    cmp al, 0
    je finalize_sum   ; End of buffer
    cmp al, '0'
    jl handle_delimiter
    cmp al, '9'
    jg handle_delimiter

    ; Convert ASCII to integer
    sub al, '0'
    movzx eax, al

    ; Multiply current number by 10 and add new digit
    mov ebx, [num]
    imul ebx, ebx, 10
    add ebx, eax
    mov [num], ebx

    jmp next_char

handle_delimiter:
    ; If we have accumulated a number, add it to sum
    mov eax, [num]
    add [sum], eax
    mov dword [num], 0   ; Reset num for next number

next_char:
    inc esi
    jmp parse_loop

finalize_sum:
```

```
    ; If there's a last number (e.g., no newline at the end)
    mov eax, [num]
    add [sum], eax

    ; Convert sum to string
    mov eax, [sum]
    mov edi, sum_str
    call int_to_str

    ; Print "Sum: "
    mov eax, 4
    mov ebx, 1
    mov ecx, sum_msg
    mov edx, 5
    int 0x80

    ; Print sum
    mov eax, 4
    mov ebx, 1
    mov ecx, sum_str
    mov edx, 12   ; Ensure we print full sum
    int 0x80

    ; Print newline
    mov eax, 4
    mov ebx, 1
    mov ecx, newline
    mov edx, 1
    int 0x80

    ; Exit
    mov eax, 1
    mov ebx, 0
    int 0x80

int_to_str:
    ; Converts integer in EAX to string in sum_str buffer
    mov ecx, 10        ; Base 10
    mov ebx, 0         ; Counter for string length
    mov edi, sum_str + 11 ; Start at end of buffer
    mov byte [edi], 0  ; Null-terminate string
    dec edi
```

```
convert_loop:
    mov edx, 0
    div ecx
    add dl, '0'
    mov [edi], dl
    dec edi
    inc ebx
    test eax, eax
    jnz convert_loop

    inc edi
    ret
```

The output:

```
[spourif1@linux5 filereadlab]$ nasm -f elf32 -g sumthefile.asm
[spourif1@linux5 filereadlab]$ ld -m elf_i386 -o tester sumthefile.o
[spourif1@linux5 filereadlab]$ tester
Sum: 4679
```

Proof of output:

Sum Calculator